向量加法指令在香山中的执行过程

基于随包归档的香山 KunminghuV2 单核完整处理器仿真波形，本报告对位于固定 PC 地址 0x80000050 处的原生向量加法指令 vadd.vv v4,v8,v9 进行了全流程追踪。实测波形表明，该指令在循环中共完成 11 次动态执行，累计产出的 176 个结果字节与软件独立计算真值 100% 吻合；在微架构流水时序上，首次执行从操作数送入执行单元到运算结果有效耗时 1 个处理器周期，从执行输入到 ROB 完成架构提交耗时 3 个处理器周期。本报告对应官方第一阶段第 1.3 项交付要求。

本报告基于随附的原始 FST 波形文件与固定版本香山源码展开剖析；文中引用的所有文件路径均以相关证明材料根目录为基准。关于波形自动复查与测试程序重汇编的操作步骤，统一归档于共用手册第 4 章；关于完整处理器重新编译与整核新波形录制流程，详见共用手册第 6 章。

1 分析对象与程序

测试程序运行时的向量控制状态设定为 VLEN=128、SEW=8、LMUL=m1、VL=16、vm=1、vstart=0，因此单条向量加法指令可并行处理 16 个相互独立的 8 位字节元素。待分析目标指令的机器编码为 0x02848257，静态链接地址固定在 0x80000050；下文将深入剖析该原生向量加法指令在香山处理器内部的硬件执行全过程。

按照 RISC-V 向量汇编语法规范，vadd.vv 的操作数排列顺序为 vd,vs2,vs1，对应本实例中的目标寄存器 vd=v4、第二源寄存器 vs2=v8、第一源寄存器 vs1=v9。硬件对每个有效字节元素分别计算两个源对应字节之和并截取低 8 位保存；相邻字节元素之间严格隔离、不传递进位，且常规整数加法不执行饱和截断处理：

```
v4[i] = (v8[i] + v9[i]) & 0xff    i = 0,1,...,15
```

指令位域	二进制取值	编码项名称与编码功能说明
31 至 26	000000	funct6，标识单宽度向量整数加法 VADD
25	1	vm，掩码禁用标志。固定为 1，不按 v0 掩码寄存器筛选
24 至 20	01000	vs2=v8，第二源向量寄存器编号
19 至 15	01001	vs1=v9，第一源向量寄存器编号
14 至 12	000	funct3=OPIVV，标识双向量源整数运算形式
11 至 7	00100	vd=v4，目标向量寄存器编号
6 至 0	1010111	opcode=0x57，RISC-V 向量算术指令主操作码

在数值解释上，例如十六进制 0x7f+0x01 截取低 8 位后为 0x80；0x7f+0x7f 求和得到 0xfe。由于本测试全程固定 VL=16 且 vm=1，128 位向量寄存器内的全部 16 个字节元素均处于有效激活状态；掩码屏蔽、尾部保留/不关心以及非零 vstart 中途重启等分支场景不在本次测试程序的覆盖范围内。

1.1 程序顺序

在测试程序的控制流设计上，启动代码首先使能处理器向量扩展状态并挂载异常向量入口。在每轮迭代开始时，程序先将目标寄存器 v4 预置为 16 个 0xa5，作为背景污染值以检验全覆盖写入，随后分别从内存装入源向量 v8 与 v9。执行 vadd.vv 完成加法后，立即调用 vs1r.v 将 128 位目标寄存器 v4 完整转存至内存缓冲区，再分两次 64 位标量双字加载与预设期望值逐一比对。一旦任一双字比对失配即跳转至 BAD 失败分支；当全部 11 组测试向量均比对通过后，最终跳转至 OK 成功分支：

```
0x80000034  li        s3,11
0x80000038  vsetivli  zero,16,e8,m1,ta,ma
0x8000003c  vle8.v    v4,(s2)
0x80000040  vle8.v    v8,(s0)
0x80000044  addi      t0,s0,16
0x80000048  vle8.v    v9,(t0)
0x8000004c  vsetivli  zero,16,e8,m1,tu,mu
0x80000050  vadd.vv   v4,v8,v9
0x80000054  vs1r.v    v4,(s1)
0x80000058  ld        t0,0(s1)
0x8000005c  ld        t1,32(s0)
0x80000060  bne       t0,t1,BAD
0x80000064  ld        t0,8(s1)
0x80000068  ld        t1,40(s0)
0x8000006c  bne       t0,t1,BAD
0x80000070  addi      s0,s0,48
0x80000074  addi      s3,s3,-1
0x80000078  bnez      s3,CASE_LOOP
0x8000007c  j         OK
```

链接脚本中将成功出口定义为 OK=0x80000088，失败出口定义为 BAD=0x80000080，异常入口设在 0x80000084，触发异常后同样跳转至 BAD。由于 OK 与 BAD 两处终点均设计为原地死循环跳转指令，仿真器在达到预设周期上限停止时停机地址恰好保持在 PC=0x80000088，证明处理器完整跑通了全部 11 组比对且未触发任何异常或失配。

1.2 数据布局

数据区符号	内存基地址与空间长度	在测试程序中的工程用途
POISON	0x80100000，16 字节	存放背景污染数据，每轮运算前将目的寄存器 v4 预置为全 0xa5
OUT	0x80100010，16 字节	作为存储缓冲区，接收 vs1r.v 转存的 128 位完整运算结果
CASES	0x80100020，11×48 字节	连续存放 11 组测试向量，每组依次排列 v8、v9 与期望结果，各占 16 字节

测试二进制镜像统一加载至物理基地址 0x80000000，代码段与数据段均按固定绝对地址链接。在字节序组织上，第 i 号元素对应 128 位向量寄存器的 [8i+7:8i] 位域；当以完整 128 位十六进制数展示时，最右侧的最低有效字节对应元素 0。在测试数据区 CASES 中，第 j 组源向量 v8 的镜像字节偏移量为 0x100020+48j。

2 运行资料与时间换算

本次原始波形录制基于 KunminghuV2Config 单核配置，分配 256 MB 模拟内存，开启完整 128 位向量寄存器 DiffTest 协同比对并启用 --no-squash 参数，底层编译采用 Verilator 5.052。随包归档的原始波形文件 waveforms/vadd.fst 体积为 67,833,959 字节，完整记录了仿真时间窗口 t=0 至 10004；配套测试镜像 vadd_full_register.bin 大小为 1,049,136 字节，汇编源码、ELF 可执行文件及反汇编清单均归档于 gen/cluster_programs 目录下。

本报告的所有源码级映射分析均严格基于随包固化的固定源码。其中，香山处理器上游固定版本 Commit 版本号为 42f87803d10bc21bbc5df2807a2fc07db6c9dc89；香山、NEMU 及各子模块的版本清单详见 source_revisions.csv，当次构建与运行参数详见 results/cluster/20261019/build_manifest.json。

为统一时序度量口径，下文以 t 表示 FST 波形文件内的原始时间戳，以 C 表示重排序缓冲区上报给 DiffTest 框架的处理器时钟周期计数。在有效采样区间 t=100 至 10004 内，每个采样点均严格满足换算关系 C=t−2 以及顶层计时器 timer=t+1。例如，首次执行时操作数送入执行单元的波形时刻 t=4314，精确对应处理器周期 C=4312。

在仿真器录波机制上，本次启用的 --dump-wave 模式固定在每个时钟周期主时钟置高（clock=1）且完成组合逻辑求值后执行信号采样。虽然 FST 文件头部的名义时间单位标注为 1ps，但每 1 个时间步进单位实际对应 1 个完整的处理器时钟周期，因此下文直接按周期差计算流水线延迟。上述采样机制与周期换算逻辑可分别在 results/vadd_wave_stage1 目录下的 source_reference/emu.cpp、source_reference/waveform.cpp、source_reference/Rob.scala 以及校准表 cycle_calibration.csv 中核实。

3 在香山中的执行路径

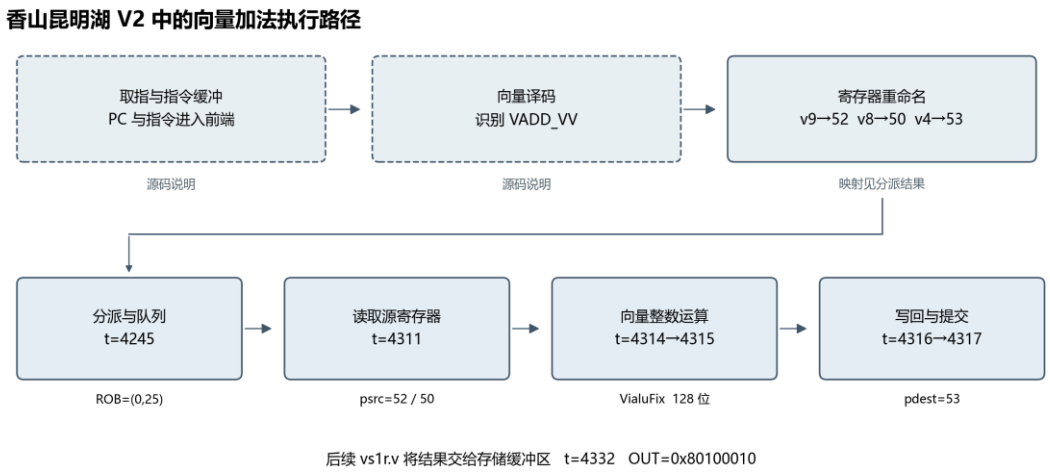


图 1 原生向量加法在香山昆明湖 V2 中的执行路径（图中标注时间均为 FST 时刻 t）

3.1 取指 译码 重命名与排队

在处理器前端，取指部件将取回的指令 PC 与 32 位指令码一并推入指令缓冲区；进入后端译码级后，向量译码表识别出 VADD_VV 指令，将其功能单元类型映射为 FuType.vialuF、子操作码设为 VialuFixType.vadd_vv，同时拉高向量寄存器写使能。根据 OPIVV 编码规则，译码器将两个源操作数类型均指定为向量物理寄存器。

随后，寄存器重命名模块将指令中的逻辑向量寄存器映射至空闲的物理向量寄存器，并为其分配 ROB 表项指针。以第 1 次动态执行为例，第一源寄存器 vs1=v9 被映射至物理寄存器 52，第二源寄存器 vs2=v8 被映射至物理寄存器 50，目标寄存器 vd=v4 被分配至新物理寄存器 53，同时分配的 ROB 索引为 (flag=0, value=25)。完成重命名后，指令进入向量发射队列等待源操作数就绪与功能单元空闲，随后由发射通路向向量物理寄存器堆发起读请求。

3.2 运算与结果去向

在后端执行通道配置上，FuConfig.scala 中的 VialuCfg 绑定了整数向量算术单元 VIAluFix，输出数据位宽设为 128 位，流水线固定延迟配置为 CertainLatency(1)。封装层 VecPipedFuncUnit.scala 将输入源端口映射为 src_0=vs1、src_1=vs2，故在本次实例中有 src_0=v9、src_1=v8。

进入 VIAluFix 算术核内部后，电路将 128 位源向量拆分为高 64 位与低 64 位两半，交由两个并行的 64 位定点算术子模块 VIntFixpAlu64b 同步运算；每个子模块各自并行处理 8 个 e8 字节元素，出拍后重新拼接为完整的 128 位结果，并统一完成掩码、尾部及 vstart 后处理逻辑。在本次配置下，指令无需拆分微操作，由上述两个 64 位计算子模块协同在单拍内完成整条 128 位加法运算。

运算产出的 128 位结果经由写回总线写入目标向量物理寄存器，并由 ROB 按程序顺序完成架构状态提交。紧随其后的向量存储指令 vs1r.v 从物理寄存器 53 读取最新更新的 v4 值，并将其推入访存存储通道；待存储缓冲区接收该 128 位数据后，后续的两条标量加载指令即可通过前递或内存读取取回结果并执行分支比对。

4 首次执行的逐阶段分析

在乱序超标量处理器波形分析中，由于循环体内的静态 PC 地址会被反复执行，且 ROB 的 value 索引号也会随队列回绕而重复出现，若仅凭单一标识过滤极易混入其他轮次的动态指令。因此，追踪同一条动态指令实例时，必须联合绑定 PC、指令机器码、完整的 ROB 标识 (flag, value) 以及 psrc/pdest 的物理寄存器编号做多维交叉匹配。

FST 时刻 t	周期 C	微架构流水事件	关键握手信号与实测关联数据
4245	4243	后端派发接收	valid=ready=1；PC=0x80000050；编码 0x02848257；ROB=(0,25)；psrc=52/50；pdest=53
4311	4309	物理寄存器读请求	valid=ready=1；携带同一 ROB 与目的寄存器编号；双源读地址分别为 52 与 50

FST 时刻 t	周期 C	微架构流水事件	关键握手信号与实测关联数据
4314	4312	算术单元执行输入	valid=ready=1；fuOpType=0x040；两个 128 位源操作数均为 16 个 0x01
4315	4313	算术运算结果有效	功能单元与执行通道输出端均满足 valid=ready=1；输出 16 个字节全为 0x02
4316	4314	物理寄存器写控制	四个 32 位分片的写端口 1 同步拉高 wen=1、addr=53、写入数据 0x02020202
4317	4315	寄存器更新与 ROB 提交	四个 32 位分片均可读出新值；ROB 提交槽位的 PC、编码、ROB 指针及 pdest 与分派完全一致
4332	4330	后继存储缓冲接收	后继 vs1r.v 触发 valid=ready=vecValid=1；写入地址 OUT=0x80100010；mask=0xffff；128 位数据一致

在首次执行的输入拍 t=4314，送入执行单元的向量控制信号实测值为 vsew=0、vlmul=0、src_4=0x10、vm=1、vstart=0、vta=vma=0、vuopIdx=0，精确对应 e8、m1、VL=16 及 tu/mu 模式。该轮输入的两个 128 位源操作数均为全 0x01（0x01010101010101010101010101010101），下一拍产出的 128 位加法结果为全 0x02（0x02020202020202020202020202020202）。

从各流水阶段的时间间隔来看，指令自派发入队 C=4243 至发起物理寄存器读请求 C=4309 共间隔 66 个周期，该时段主要消耗在等待前置向量加载指令 vle8.v 数据就绪及队列调度上；从发出寄存器读请求到数据正式送入执行单元 C=4312 间隔 3 个周期；而从执行单元采样输入到计算结果有效 C=4313 仅间隔 1 个周期，与 VialuCfg 中 CertainLatency(1) 的硬件延迟设定完全吻合。



图 2 128 位完整运算结果与四个 32 位物理寄存器分片写回的对应关系

在香山向量物理寄存器堆底层实现中，单个 128 位向量寄存器按位域拆分为四个 32 位分片独立存储。在 t=4316 时刻，四个分片的写端口 1 同步拉高写使能，wen=1、addr=53、写入值均为 0x02020202；在紧接着的下一个采样拍 t=4317，即可从物理寄存器 53 的四个分片中读出更新后的新值。按 Part 3 居最高位、Part 0 居最低位的顺序拼接后，即还原出完整的 128 位结果。综上，算术结果有效、寄存器写控制有效与寄存器内容完成更新的波形时刻依次对应 t=4315、t=4316 和 t=4317。

与此同时，ROB 在 t=4317 时刻的有效提交槽位 commitValid_0=1 所携带的 PC、指令编码、ROB 编号 (0,25) 及目的物理寄存器号 pdest=53 与分派时刻完全一致，确认当拍完成架构提交的正是本次 vadd.vv 指令。随后的 t=4332 时刻则对应后继存储指令 vs1r.v 将结果写入存储缓冲区的时刻；从 vadd.vv 执行输入到 Sbuffer 接收数据之间的 18 个周期包含了后继存储指令本身的流水开销，而 vadd.vv 自身的算术运算延迟仍为 1 个周期。

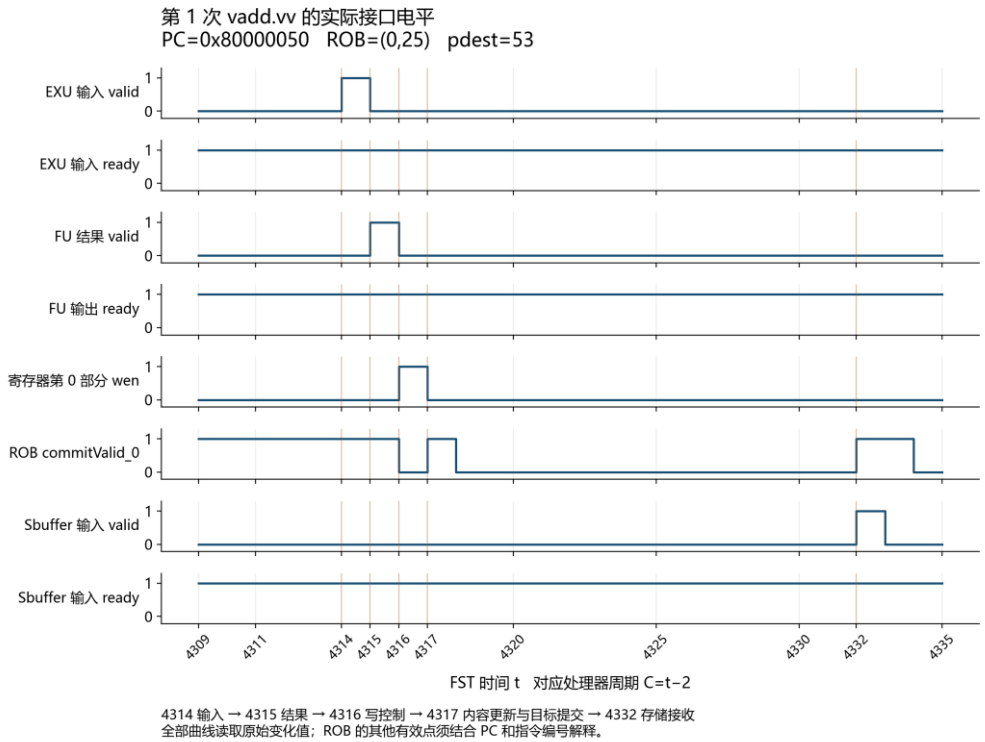


图 3 首次执行期间关键硬件接口信号实测波形（ROB 其他高电平脉冲需结合 PC 与指令编号甄别）

5 全部执行结果

在全部 11 轮测试中，第 1 轮的 16 个字节均执行相同的 0x01+0x01=0x02 运算；为验证不同字节通道的独立性与位置对齐，可重点考察各字节互异的第 6 轮测试：当按元素索引 0 至 15 顺序展开时，两个源向量与目标向量的实测字节序列如下：

```
v8: 03 3d 0c 63 07 2d 02 64 00 00 00 00 00 00 00 00
v9: 05 04 03 02 01 5a 40 0b 00 00 00 00 00 00 00 00
v4: 08 41 0f 65 08 87 42 6f 00 00 00 00 00 00 00 00
```

举例而言，第 1 号元素执行十进制 61+4=65（对应 0x3d+0x04=0x41）；第 5 号元素执行十进制 45+90=135（对应 0x2d+0x5a=0x87，作为有符号字节解释时为 −121）。拼接后的 128 位完整寄存器十六进制值为 0x0000000000000006f428708650f4108。

执行轮次	ROB 编号	目的物理寄存器	执行输入 C	结果有效 C	写控制 C	架构提交 C	Sbuffer 接收 C
1	(0,25)	53	4312	4313	4314	4315	4330
2	(0,42)	61	4386	4387	4388	4389	4404
3	(0,59)	69	4446	4447	4448	4449	4464

执行轮次	ROB 编号	目的物理寄存器	执行输入 C	结果有效 C	写控制 C	架构提交 C	Sbuffer 接收 C
4	(0,76)	77	4549	4550	4551	4552	4567
5	(0,93)	85	4610	4611	4612	4613	4628
6	(0,110)	93	4670	4671	4672	4673	4688
7	(0,127)	101	4730	4731	4732	4733	4748
8	(0,144)	109	4790	4791	4792	4793	4808
9	(1,1)	117	4851	4852	4853	4854	4869
10	(1,18)	125	4911	4912	4913	4914	4929
11	(1,35)	9	5015	5016	5017	5018	5033

执行轮次	128 位完整向量实测写回结果	逐字节比对结论
1	0x02020202020202020202020202020202	16/16 字节完全一致
2	0x000000000000000100e0c0a08060402	16/16 字节完全一致
3	0x0000000000000000fefefefefefefefe	16/16 字节完全一致
4	0x00000000000000008080808080808080	16/16 字节完全一致
5	0x00000000000000000909090909090909	16/16 字节完全一致
6	0x0000000000000006f428708650f4108	16/16 字节完全一致
7	0x0000000000000000c759afd3bac2b115	16/16 字节完全一致
8	0x0000000000000000a2817f8d8fa94faf	16/16 字节完全一致
9	0x0000000000000006d162fcd96588738	16/16 字节完全一致
10	0x00000000000000000775fea7ac5bd5576	16/16 字节完全一致
11	0x00000000000000000996b9c6b5758a2b7	16/16 字节完全一致

汇总上表可见，全部 11 次动态执行的硬件流水节奏高度一致：从执行输入到结果有效均固定为 1 个周期，从执行输入到 ROB 架构提交均固定为 3 个周期，从执行输入到后继存储指令送达 Sbuffer 均固定为 18 个周期。在第 9 轮执行时，ROB 队列发生回绕，编号变为 (1,1)；在第 11 轮执行时，目的物理寄存器编号回绕分配为 9；依靠完整的四元组标识依然能够精准锁定每次动态执行。11 轮累计产出的 176 个结果字节与期望真值 100% 吻合。

6 配套材料与复查入口

文件相对路径	文件内容与工程复核用途
waveforms/vadd.fst	在香山单核完整处理器上录制的原始 FST 波形文件
gen/cluster_programs/vadd_full_register.*	原生向量加法测试程序的汇编源码、二进制镜像、ELF 可执行文件及反汇编清单
results/vadd_wave_stage1/signals.csv	波形提取所涉及的 260 个内部硬件信号完整层次路径、位宽定义及 RTL 源码位置
results/vadd_wave_stage1/extract_all.csv	从原始波形提取的全量信号跳变记录表，供离线复算各统计表格

文件相对路径	文件内容与工程复核用途
results/vadd_wave_stage1/vadd_exec1_events.csv	第 1 次动态执行各流水阶段的关键时刻、控制信号与操作数明细表
results/vadd_wave_stage1/vadd_all11_summary.csv	全部 11 次动态执行的流水阶段周期时刻与 128 位完整结果汇总表
results/vadd_wave_stage1/element_checks.csv	11 轮累计 176 个字节元素的双源输入、理论期望值与硬件实测值逐项核对表
results/vadd_wave_stage1/cycle_calibration.csv	FST 采样时间戳 t、ROB 处理器周期 C 与顶层 timer 的逐拍对齐校准表
results/vadd_wave_stage1/source_reference 与 reference	支撑本报告微架构执行路径与译码逻辑分析的固化固定源码切片

reproduce_vadd.py 直接读取原始 FST 并复算各统计表，4 类错误输入处理检查通过。动态指令按 PC、机器码、完整 ROB 编号及源目的物理寄存器编号匹配。

波形复查与表格重算见共用手册第 4.2 节，完整处理器新波形录制见第 6.2 节，工具准备见第 2 章。

7 参考资料与关键源码

vadd.vv 的算术语义与向量控制规则采用 RISC-V Vector 1.0 规范；下表源码位置对应随附的香山版本。

```
https://docs.riscv.org/reference/isa/unpriv/v-st-ext
https://github.com/OpenXiangShan/XiangShan
```

关键源码文件名	对应代码行号区间及微架构功能说明
IBuffer.scala	第 66 行起，指令缓冲区定义，缓存前端取回的指令 PC 与 32 位指令码
RocketInstructions.scala	第 407 行，定义 VADD_VV 机器编码的位域匹配规则
VecDecoder.scala	第 189 行及 OPIVV 规则定义，映射向量源操作数类型与目标功能单元
FuConfig.scala	第 532 至 550 行，配置 VialuCfg 的 128 位数据位宽与 CertainLatency(1) 单周期延迟
VecPipedFuncUnit.scala	第 61 至 66 行，绑定流水功能单元输入端口 src_0=vs1、src_1=vs2
VIAluFix.scala	第 121、134、235、275 行起，实现双 64 位子模块并行计算、结果拼合与掩码尾部处理
Regfile.scala	第 119、255 行起，实现向量物理寄存器堆的四分片独立写入与读取拼接
Rob.scala	第 1607 行，定义重排序缓冲区上报给 DiffTest 的处理器周期计数来源
emu.cpp 与 waveform.cpp	第 330 至 380 行及第 59 行起，定义仿真器时钟置高求值后的波形采样机制与时间戳步进